개인용 macOS 개발환경 이전 앱 문서 팩

PRD / UX 명세 / 기술 설계 / QA 계획 / 로드맵

버전 0.9 draft

작성일: 2026-03-02

대상: 개인용 로컬 전용 macOS 앱

문서 목적	**새 맥에서 기존 개발환경을 안전하게 재현하기 위한 제품 정의와 개발 착수 문서를 한 번에 제공한다.**
권장 구현 방향	SwiftUI 기반 macOS 앱 + 로컬 manifest JSON + CLI/파일 어댑터 오케스트레이션
핵심 원칙	local-first, explicit consent, resumable import, safe defaults, visible verification
기본 가정	개인 사용자가 자신의 맥 두 대 사이에서 개발환경을 이전한다. 서버, 계정 시스템, 팀 협업 기능은 제외한다.

문서 구성

이 문서 팩은 실제 개발 착수에 바로 사용할 수 있도록 다음 내용을 하나의 패키지로 묶었다. 별도 분리 문서로 재구성하기 쉽게 섹션별로 독립성을 유지했다.

- 1. 제품 개요와 문제 정의
- 2. PRD: 목표, 범위, 요구사항, 성공 지표
- 3. UX 명세: 화면, 플로우, 상태, 에러/권한 처리
- 4. 기술 설계: 아키텍처, 모듈, 데이터 모델, restore 순서
- 5. QA 계획: 테스트 전략, 시나리오, 승인 기준
- 6. 실행 로드맵: 주차별 계획, 산출물, 리스크
- 7. 부록: sample manifest, 참고자료

사용 전제와 범위 고정

이 문서 팩은 '맥 전체를 복제하는 앱'이 아니라 '새 맥에서 개발환경을 재현하는 앱'을 목표로 한다. Apple 의 Migration Assistant 는 문서, 앱, 사용자 계정, 설정을 이전할 수 있지만 운영체제를 대체하지는 않으며 [R1], 본 제품은 그 위에 개발자 관점의 선언형 재현성과 검증 기능을 제공하는 별도 도구로 정의한다.

범위 축소를 위해 v1 의 필수 지원 대상을 Homebrew, dotfiles allowlist, Git global config, VS Code 설정 및 확장으로 제한한다. SSH 키, 로그인 세션, 브라우저 쿠키, Keychain, DB 데이터 전체 복제는 기본 제외로 둔다. 다만 사용자가 나중에 직접 켜서 추가할 수 있는 확장 포인트는 설계에 반영한다.

1. 제품 개요

1.1 문제 정의

새 맥을 구매한 개발자는 보통 Homebrew 재설치, shell 설정 복원, Git identity 설정, IDE 확장 복원, 언어 런타임 설치, 재로그인, 수동 권한 승인 같은 작업을 반복한다. 이 과정은 시간이 오래 걸리고, 무엇이 누락됐는지 확인하기 어렵고, 같은 실수를 반복하기 쉽다.

개발자에게 필요한 것은 단순 파일 복사가 아니라 '내가 의도한 개발환경 상태'를 기록하고 새 맥에서 같은 상태에 최대한 가깝게 복원한 뒤, 무엇이 자동으로 되었고 무엇이 수동으로 남았는지를 명확하게 보여주는 경험이다. Homebrew Bundle 이 선언형 설치 상태를 표현하는 방식 [R2], Git 설정의 scope 구조 [R5], VS Code 의 CLI 기반 확장 관리 [R6]는 이 제품의 핵심 구현 기반이 된다.

1.2 제품 비전

한 번의 export 로 현재 맥의 개발환경을 이해 가능한 manifest 와 패키지로 보존하고, 새 맥에서는 import 한 번으로 동일한 개발 준비 상태에 도달하게 한다. 사용자는 복원 후 '무엇이 설치되었는지, 무엇이 스킵되었는지, 무엇을 수동으로 해야 하는지'를 리포트로 확인할 수 있어야 한다.

1.3 제품 원칙

- Local-first: 사용자 데이터와 패키지는 기본적으로 로컬 디스크에만 저장한다.
- Safe defaults: secret 와 파괴적 동작은 기본 제외하고 명시적 opt-in 으로만 허용한다.
- Deterministic restore: import 순서와 결과가 기록 가능해야 하며 중단 후 재개가 가능해야 한다.
- Visible automation: 백그라운드에서 몰래 진행하지 않고 단계, 로그, 스킵 사유를 모두 보여준다.
- Adaptors over hardcoding: Homebrew, Git, VS Code 같은 개별 도구는 어댑터로 분리한다.

1.4 성공한 상태의 정의

항목	성공 정의
개발 준비 시간	새 맥 초기화 후 앱 실행부터 주요 프로젝트 clone 및 첫 build 가능 상태까지의 시간을 기존 수작업 대비 70% 이상 단축
재현성	manifest 에 포함된 항목 기준 90% 이상이 자동 또는 준자동으로 복원
가시성	모든 스킵/실패 항목에 대해 원인과 다음 액션이 리포트에 남음
안전성	기본 설정에서 secret 와 시스템 민감 영역을 자동 복사하지 않음

2. PRD

2.1 목표와 비목표

구분	내용
목표	개발환경 export, import, verify 의 3 단계 경험을 하나의 macOS 앱으로 제공
목표	Homebrew 패키지/앱/서비스, dotfiles allowlist, Git global config, VS Code 설정/확장을 안정적으로 재현
목표	중단 후 재개와 검증 리포트 제공
비목표	macOS 전체 백업 또는 전체 디스크 복제
비목표	로그인 세션, Keychain, 브라우저 쿠키, 앱 라이선스의 완전 자동 이전
비목표	팀 협업, 클라우드 동기화, 중앙 관리 콘솔

2.2 핵심 페르소나

페르소나는 '개인 프로젝트와 업무 프로젝트를 동시에 다루는 1 인 개발자'다. 그는 새 맥을 받으면 빠르게 기존 생산성을 회복해야 하고, 반복 작업을 자동화하고 싶지만, 어떤 파일이 안전하게 복사 가능한지와 어떤 도구가 OS 권한이나 재로그인을 요구하는지까지 모두 기억하고 싶어 하지는 않는다.

그는 자신이 사용하는 도구를 알고 있으므로, 앱은 과한 마법보다 명시적인 리뷰와 선택 가능한 restore plan 을 제공해야 한다. 특히 로컬 데이터 프라이버시를 중요하게 여기므로, 네트워크 연결 없이 동작하는 구조가 선호된다.

2.3 JTBD

- When I buy a new Mac, I want to reproduce my development environment from a local package so that I can get back to coding in one sitting.
- When I restore to a new machine, I want to know exactly what was applied, skipped, or needs manual action so that I do not waste time debugging invisible setup gaps.
- When I update my environment over time, I want export artifacts to stay readable so that I can audit and refine what the app manages.

2.4 범위 정의

범주	v1 상태	설명
Homebrew formula / cask / tap / service	포함	Brewfile 중심으로 export/import. prefix 와 architecture 정보를 함께 저장 [R2] [R3].
dotfiles allowlist	포함	~/.zshrc, ~/.gitignore_global, ~/.config/* 등 사용자가 명시한 경로만 포함.

범주	v1 상태	설명
Git global config	포함	--global 범위만 처리. local/worktree/repo 설정은 제외 [R5].
VS Code settings / keybindings / snippets / extensions	포함	CLI 기반 확장 목록과 사용자 설정 파일 복원 [R6].
Xcode CLT 감지	포함	없으면 사전 체크 단계에서 수동 설치 안내.
JetBrains IDE	후속	v1 에서는 어댑터만 정의하고 실제 export/import 는 추후 추가.
SSH 키 / GPG 키	기본 제외	보안상 v1 기본 제외. 추후 opt-in 설계.
Docker volume / 로컬 DB 데이터	기본 제외	정합성, 크기, 중단/재개 복잡도 때문에 후속 단계.
Keychain / iCloud Keychain	제외	OS 및 Apple 계정 동기화 영역으로 본 제품 책임 밖 [R7].

2.5 사용자 스토리

ID	사용자 스토리	우선순위
US-01	사용자로서 현재 맥의 개발환경을 스캔하고 export package 로 저장하고 싶다.	P0
US-02	사용자로서 export 전에 어떤 항목이 포함/제외되는지 검토하고 싶다.	P0
US-03	사용자로서 새 맥에서 package 를 열면 설치 전 preflight 결과를 보고 싶다.	P0
US-04	사용자로서 import 가 중단되더라도 다음 실행에서 이어서 진행하고 싶다.	P0
US-05	사용자로서 restore 후 누락된 수동 단계와 검증 결과를 한 화면에서 확인하고 싶다.	P0
US-06	사용자로서 dotfiles 경로를 allowlist 로 관리하고 싶다.	P1
US-07	사용자로서 VS Code 외 IDE 지원을 어댑터 단위로 나중에 추가하고 싶다.	P1

2.6 기능 요구사항

ID	요구사항	수용 기준 요약	우선순위
FR-01	앱은 source Mac 의 OS 버전, CPU architecture, Homebrew prefix, 로그인 사용자 홈 경로를 수집해야 한다.	export report 에 machine metadata 가 기록된다.	P0
FR-02	앱은 export 전 스캔 결과를 카테고리별로 보여주고 include/exclude 를 조정하게 해야 한다.	사용자가 항목별 체크 상태를 수정할 수 있다.	P0
FR-03	앱은 Homebrew 상태를 선언형 manifest 로 저장해야 한다.	formula/cask/tap/service 가 JSON 과 Brewfile 형태로 저장된다.	P0
FR-04	앱은 사용자 지정 dotfiles allowlist 를 파일 패키지에 복사해야 한다.	허용 경로만 복사되고 스킵 항목 사유가 기록된다.	P0
FR-05	앱은 Git global config 를 export/import 해야 한다.	user.name, user.email 등 global 설정이 새 맥에 적용된다.	P0

ID	요구사항	수용 기준 요약	우선순위
FR-06	앱은 VS Code 확장 목록과 사용자 설정 파일을 export/import 해야 한다.	확장 재설치와 settings.json 복원이 가능하다.	P0
FR-07	앱은 secret 후보를 자동 분류하고 기본 제외해야 한다.	SSH, keychain, browser profiles 는 기본 제외 목록에 나타난다.	P0
FR-08	앱은 export 결과를 하나의 package 폴더 또는 압축 파일로 저장해야 한다.	manifest.json, files/, reports/ 구조가 생성된다.	P0
FR-09	앱은 target Mac 에서 preflight 를 수행해 누락된 조건을 알려야 한다.	예: Xcode CLT 미설치, brew 미존재, disk space 부족.	P0
FR-10	앱은 bootstrap -> packages -> configs -> ide -> verify 순서로 restore 를 수행해야 한다.	restore plan 과 실제 실행 로그가 일치한다.	P0
FR-11	앱은 Rosetta, logout, reboot 같은 수동 단계 후보를 task queue 로 보여줘야 한다.	manual tasks 화면이 생성된다 [R4].	P1
FR-12	앱은 중단 시 checkpoint 를 저장하고 다음 실행에서 resume 해야 한다.	성공 단계는 다시 실행되지 않거나 idempotent 하게 처리된다.	P0
FR-13	앱은 dry-run 모드를 제공해야 한다.	실제 파일 변경 없이 restore plan 만 계산해 보여준다.	P1
FR-14	앱은 verify 단계에서 command-level 검증을 수행해야 한다.	brew doctor, git config, code --list-extensions 등 핵심 검증 결과가 리포트에 남는다.	P0
FR-15	앱은 사람이 읽을 수 있는 최종 리포트를 생성해야 한다.	자동 성공/수동 필요/실패가 구분된 HTML 또는 Markdown 리포트가 생성된다.	P0

2.7 비기능 요구사항

ID	요구사항	설명
NFR-01	Local-only 기본값	기본 동작에서 외부 서버 전송 없음. 네트워크는 brew/extension install 등 사용자가 명시적으로 import 를 실행할 때만 필요.
NFR-02	설명 가능성	모든 자동화 단계는 로그와 restore plan 으로 설명 가능해야 한다.
NFR-03	회복 가능성	실패가 전체 과정을 망치지 않고 부분 재시도와 resume 가 가능해야 한다.
NFR-04	안전한 쓰기	기존 파일 덮어쓰기는 backup 또는 .bak 정책을 거친다.
NFR-05	권한 최소화	가능하면 파일 읽기/CLI 호출만 사용하고 Accessibility/Automation 권한은 v1 에서 요구하지 않는 것을 목표로 한다 [R8].
NFR-06	읽기 쉬운 산출물	manifest.json 과 report 파일은 사람이 직접 열어도 이해할 수 있어야 한다.
NFR-07	호환성 전략	동일 architecture 경로를 우선 지원하고, 교차 architecture 는 명시적 경고와 함께 처리한다 [R3].
NFR-08	성능	중간 규모 환경 기준 export 스캔은 3 분 이내, manifest 생성은 30 초 이내를 목표로 한다.

2.8 성공 지표

지표	정의	초기 목표
Export completeness	사용자가 기대한 관리 대상 중 export package 에 포함된 비율	90%+
Import completion	P0 항목 중 자동 또는 준자동으로 완료된 비율	85%+
Manual task clarity	수동 단계 항목 중 다음 액션이 명확히 적힌 비율	100%
Resume reliability	중단 후 재실행 시 손상 없이 재개한 세션 비율	95%+

2.9 가정, 제약, 의존성

- Homebrew Bundle 은 선언형 상태 복원에 적합하지만, 패키지 생태계 특성상 과거 시점의 완전 고정 재현까지 자동 보장하지는 않는다. 따라서 앱은 manifest 에 버전과 당시 metadata 를 최대한 저장하되, 'best-effort reproducibility'를 제품 약속으로 둔다 [R2].
- Homebrew 기본 prefix 는 Apple silicon 과 Intel 에서 다르므로, machine metadata 와 restore preflight 가 필수다 [R3].
- macOS 는 Full Disk Access, Accessibility, Automation 등 민감 권한에 대해 사용자 승인을 요구한다. v1 은 가능한 범위에서 이 권한들을 피하도록 설계한다 [R8].
- Migration Assistant 와 역할이 일부 겹치지만, 본 제품은 '개발환경 선언/복원/검증'에 집중한다 [R1].

3. UX 명세

3.1 주요 화면 목록

화면	목적	핵심 요소
Home	Export/Import 진입	최근 package, 마지막 리포트, 새 export 버튼
Export Scan	현재 환경 스캔	카테고리 카드, 진행률, 스킵 이유
Export Review	포함/제외 최종 확인	카테고리별 item count, allowlist 편집, package 저장 위치
Import Preflight	대상 맥 조건 확인	OS/arch 체크, brew/CLT 상태, disk space, 충돌 파일 예고
Import Run	복원 실행	phase progress, live logs, step retry, pause/cancel
Manual Tasks	사용자 개입 단계 안내	relogin/reboot/Rosetta/권한 승인 체크리스트
Verify Report	최종 검증	성공/스킵/실패 요약, 명령 결과, 다음 액션

3.2 핵심 사용자 플로우

Export 플로우

1. 사용자가 Home 화면에서 New Export 를 선택한다.
2. 앱은 시스템 정보와 지원 어댑터(Homebrew, dotfiles, Git, VS Code)를 스캔한다.
3. 스캔 결과를 카테고리 카드로 보여주고, 사용자는 include/exclude 와 경로 allowlist 를 조정한다.
4. 사용자가 저장 위치를 고르면 앱은 manifest, 파일 복사본, 리포트를 package 로 생성한다.
5. 완료 화면에서 package 경로와 export summary 를 보여준다.

Import 플로우

6. 사용자가 package 를 선택한다.
7. 앱은 target Mac 에서 preflight 를 실행하고 위험 요소를 보여준다.
8. 사용자가 실제 import 또는 dry-run 을 선택한다.
9. 앱은 restore phase 순서대로 작업을 수행하고 각 단계 결과를 checkpoint 에 기록한다.
10. 필요한 수동 단계가 있으면 Manual Tasks 화면으로 전환한다.
11. 검증이 끝나면 Verify Report 를 생성하고 사용자는 세부 로그를 열람한다.

3.3 UX 원칙

- 마법보다 예측 가능성: 자동화 자체보다 '무엇을 하는지'를 명확히 보여준다.
- 수동 단계도 제품 일부: 사용자 개입이 필요한 순간을 실패가 아니라 설계된 경험으로 다룬다.
- 중단 가능성 전제: 긴 설치 과정에서 앱 종료나 네트워크 문제를 정상 경로로 취급한다.
- 선택 가능한 범위: 전체 복원과 선택적 복원을 모두 지원하되 기본은 추천 플랜을 제시한다.

3.4 권한 및 보안 UX

v1 은 가능한 한 파일 읽기와 CLI 호출로 해결하고, UI 자동화나 접근성 권한을 전제로 하지 않는다. 사용자가 특정 경로를 export 대상으로 추가했지만 OS 정책상 접근이 제한되면, 앱은 해당 항목을 실패로 숨기지 않고 '권한 필요 - 수동 복사 권장' 상태로 표시한다. macOS 는 전체 저장장치 접근, 접근성, 자동화 기능에 사용자 승인을 요구하므로 [R8], 권한 요청은 최초 필요 시점에만 보여주는 것이 좋다.

3.5 상태 정의

상태	설명	사용자에게 보이는 메시지 예시
Detected	스캔으로 발견됨	환경에서 발견됨
Included	export/import 대상에 포함	이 항목은 package 에 포함됩니다
Excluded	정책 또는 사용자 선택으로 제외	기본 제외 항목입니다. 필요 시 직접 추가하세요
Blocked	권한 또는 시스템 제약으로 자동 처리 불가	자동 복원 불가. 수동 단계가 필요합니다
Applied	복원 성공	복원 완료
NeedsManualAction	사용자 액션 대기	로그아웃 후 다시 열어 계속 진행하세요
Failed	실패	세부 로그를 확인하고 재시도하세요

3.6 마이크로카피 가이드

- 권한 관련 문구는 '허용하지 않으면 사용할 수 없음'보다 '허용하면 자동화 범위가 넓어짐' 방식으로 표현한다.
- 실패 문구는 모호한 오류 코드 대신 action-oriented 문장으로 쓴다. 예: 'Xcode Command Line Tools 가 없어 Homebrew restore 를 시작할 수 없습니다.'
- 수동 단계 문구는 체크리스트와 예상 결과를 함께 보여준다. 예: 'Rosetta 설치 후 이 화면으로 돌아와 재시도하세요.'

4. 기술 설계

4.1 권장 아키텍처

권장 구현은 SwiftUI 기반 macOS 앱 셸 위에 어댑터 기반 오케스트레이션 레이어를 두는 구조다. 네이티브 앱은 권한, 파일 시스템, 프로세스 실행, Finder 연동에서 유리하고, 로컬 전용 제품과도 잘 맞는다. 어댑터는 Homebrew, Git, VS Code 같은 외부 도구별 수집/복원/검증 로직을 캡슐화한다.

```
App Shell (SwiftUI)
  -> Scan Orchestrator
```

```
      -> HomebrewAdapter
      -> DotfilesAdapter
      -> GitAdapter
      -> VSCodeAdapter
   -> Manifest Engine
   -> Package Builder
   -> Import Runner
   -> Verify Runner
   -> Report Generator
   -> Checkpoint Store
```

4.2 모듈 책임

모듈	책임	비고
Scan Orchestrator	어댑터 실행 순서 관리, progress 통합, 스캔 실패 누적	읽기 전용
Manifest Engine	정규화된 item 모델 생성, JSON 직렬화, schema 검증	핵심 도메인
Package Builder	files/ 디렉터리 생성, manifest/report 저장, 압축 옵션 처리	로컬 파일 시스템
Import Runner	phase 실행, 충돌 처리, checkpoint 저장	idempotent 설계 필요
Verify Runner	설치 결과 확인, command 기반 검증	리포트 입력 생성
Report Generator	사람이 읽는 최종 산출물 생성	Markdown 또는 HTML 권장
Policy Engine	secret 제외, overwrite 정책, risky action 승인 여부 판단	보안 경계

4.3 데이터 모델

필드	타입	설명
schemaVersion	string	manifest 포맷 버전. 예: 1.0.0
exportedAt	datetime	export 생성 시각
machine	object	source machine metadata: hostname, arch, macOS, home path, brew prefix
items[]	array	복원 가능한 최소 단위 목록
restorePlan[]	array	phase 순서와 각 item 의 action 요약
manualTasks[]	array	자동화 불가 또는 추후 사용자 액션 필요 항목
reports	object	scan summary, verify summary, warnings

4.4 Item 단위 모델

필드	예시	설명
id	brew.formula.git	안정적인 item 식별자
kind	brew_formula / dotfile / git_global / vscode_extension	어댑터별 타입
title	Git	화면 노출명
restorePhase	packages	bootstrap, packages, config, ide, verify 중 하나
source	{"path": "~/.gitconfig"}	원본 정보
payload	{"name": "git", "version": "2.x"}	복원에 필요한 정규화 데이터
secret	false	기본적으로 사용자 승인 없이는 true 항목 처리 금지
risk	low / medium / high	덮어쓰기 및 시스템 영향도
verify	{"command": "git config --global --get user.email"}	검증 방식

4.5 package 디렉터리 구조

```
 MyDevEnvExport/
```

```
    manifest.json
    manifest.schema.version.txt
    files/
      dotfiles/
        .zshrc
        .gitignore_global
      vscode/
        settings.json
        keybindings.json
        snippets/
    reports/
      export-summary.md
      verify-summary.md
    checkpoints/
      import-state.json
```

4.6 restore phase 정의

순서	phase	설명
0	preflight	target machine 조건 검사: architecture, disk space, Xcode CLT, brew 존재 여부
1	bootstrap	필수 도구 설치 또는 안내. 예: Homebrew 설치, code CLI 확인 [R3].
2	packages	brew formula/cask/tap/service 복원 [R2].
3	config	dotfiles, Git global config, shell 설정 복원 [R5].
4	ide	VS Code 설정과 확장 복원 [R6].
5	manual	로그아웃, 재부팅, Rosetta, 권한 승인 등 사용자 액션 목록 표시 [R4].
6	verify	명령 실행 및 파일 존재 검증, 최종 리포트 생성.

4.7 어댑터별 구현 메모

HomebrewAdapter

- 현재 설치 상태를 Brewfile 과 JSON 모두로 저장한다. Brewfile 은 선언형 restore 입력이고, JSON 은 UI 표시와 검증에 사용한다.
- machine metadata 에 brew prefix 를 저장해 target Mac 의 prefix 와 다르면 경고한다. Homebrew 기본 prefix 는 Apple silicon 에서 /opt/homebrew, Intel 에서 /usr/local 이다 [R3].
- cask metadata 로부터 Rosetta, logout, reboot 같은 manual task 후보를 생성한다 [R4].

DotfilesAdapter

- 기본 allowlist 예시: ~/.zshrc, ~/.zprofile, ~/.gitconfig, ~/.gitignore_global, ~/.config, ~/.ssh/config(파일만, private key 제외).
- wildcard 남용을 막기 위해 디렉터리 단위 포함은 .config 같이 명시적 경로에만 허용한다.
- 복원 시 덮어쓰기 전에 .bak 타임스탬프 사본을 남긴다.

GitAdapter

- v1 은 global scope 만 다룬다. system, local, worktree 설정은 read-only 로 감지하더라도 복원 대상에서 제외한다 [R5].
- 검증은 git config --global --list 결과와 manifest payload 를 비교하는 방식으로 수행한다.

VSCodeAdapter

- 확장 목록은 code --list-extensions --show-versions 출력으로 수집한다 [R6].
- 설정 파일은 settings.json, keybindings.json, snippets 디렉터리 중심으로 관리한다.

- v1 은 profile 별 동기화보다 default user profile 복원을 우선한다.

4.8 충돌 및 오류 처리 전략

상황	처리 전략
대상 파일 이미 존재	diff 요약을 보여주고 overwrite / keep both / skip 중 선택
Homebrew 설치 없음	preflight 에서 bootstrap 단계 안내 또는 자동 설치 선택지 제공
CLI 실행 실패	stderr 와 exit code 를 checkpoint 및 리포트에 저장
중간 종료	마지막 성공 phase 와 item cursor 를 저장해 resume
권한 부족	Blocked 상태로 전환하고 수동 경로 안내

4.9 보안 모델

- 기본 정책은 secret-free export 다. SSH private key, Keychain 항목, 브라우저 프로필, 토큰 저장소는 자동 제외한다.
- 패키지는 기본적으로 사용자 지정 폴더에 생성하며, 앱 내부에 장기 보관하지 않는다.
- 리포트와 로그는 민감 값을 마스킹한다. 예: user.email 일부, 토큰 패턴, 개인 경로 세부값.
- v2 이후 선택 과제로 패키지 암호화(zip with password 또는 앱 내부 암호화)를 검토한다.
- Apple 의 iCloud Keychain 은 별도의 동기화 메커니즘을 제공하므로 [R7], 본 제품은 그 내부 데이터를 직접 다루지 않는다.

4.10 sample manifest

```
{
  "schemaVersion": "1.0.0",
  "exportedAt": "2026-03-02T10:30:00+09:00",
  "machine": {
    "hostname": "old-macbook-pro",
    "architecture": "arm64",
    "macosVersion": "15.x",
    "homebrewPrefix": "/opt/homebrew"
  },
  "items": [
    {
      "id": "brew.formula.git",
      "kind": "brew_formula",
      "title": "git",
      "restorePhase": "packages",
      "payload": {"name": "git"},
      "secret": false
    },
    {
      "id": "dotfile.zshrc",
      "kind": "dotfile",
      "title": "~/.zshrc",
      "restorePhase": "config",
      "source": {"path": "~/.zshrc"},
      "secret": false
    },
    {
      "id": "git.global.user.email",
      "kind": "git_global",
      "title": "git user.email",
      "restorePhase": "config",
      "payload": {"key": "user.email", "value": "masked@example.com"},
      "secret": false
    }
  ]
}
```

5. QA 계획과 승인 기준

5.1 테스트 전략

- 단위 테스트: manifest serialization, path normalization, overwrite policy, checkpoint recovery.
- 통합 테스트: HomebrewAdapter, GitAdapter, VSCodeAdapter 를 fixture 기반으로 검증.
- E2E 테스트: export -> transfer -> import -> verify 전체 흐름을 실제 또는 가상 환경에서 검증.
- 회귀 테스트: 새 어댑터 추가 시 기존 package 포맷과 resume 동작이 깨지지 않는지 확인.

5.2 우선 시나리오 매트릭스

TC	시나리오	기대 결과
TC-01	동일 architecture 맥 간 export/import	P0 범위 복원 성공, verify report green
TC-02	교차 architecture 맥 간 import	prefix 차이 경고 표시, 가능한 항목만 복원
TC-03	target Mac 에 Homebrew 미설치	preflight 경고 후 bootstrap 진행
TC-04	target Mac 에 Xcode CLT 미설치	Homebrew restore 전 수동 단계로 차단
TC-05	VS Code 미설치	설치 안내 또는 관련 단계 스킵 후 명확한 리포트 생성
TC-06	dotfile 충돌	backup 생성 후 overwrite/skip 정책 동작
TC-07	import 중 앱 강제 종료	resume 시 마지막 checkpoint 부터 재개
TC-08	권한 부족 경로 포함	Blocked 상태와 수동 안내가 기록
TC-09	cask 중 Rosetta 필요 항목 존재	manual task queue 에 Rosetta 설치 단계 생성 [R4]
TC-10	invalid package	사용자가 이해 가능한 오류와 복구 방법 제시

5.3 승인 기준

영역	승인 기준
Export	P0 범주를 스캔하고 package 를 생성할 수 있어야 한다.
Import	동일 architecture 환경에서 P0 범주 85% 이상 자동 또는 준자동 복원
Resume	강제 종료 후 재실행 시 중복 설치/손상 없이 재개
Report	실패/스킵/수동 단계가 누락 없이 표시
Safety	기본 설정에서 secret 가 package 에 포함되지 않음

5.4 릴리스 체크리스트

- P0 기능 완료 및 회귀 테스트 통과
- 대표 시나리오 10 건 E2E 통과
- resume 및 overwrite backup 검증 완료
- 문서화: README, known limitations, supported scope 최신화
- 개인용 배포 경로 확인: local build 또는 signed app bundle 중 선택

6. 실행 로드맵

6.1 권장 마일스톤

주차	목표	산출물
1 주차	도메인 모델 확정	manifest schema, package layout, scope freeze
2 주차	Export 코어	Scan Orchestrator, Homebrew export, dotfiles allowlist export

주차	목표	산출물
3 주차	Import 코어	preflight, restore phase runner, checkpoint store
4 주차	Git + VS Code 어댑터	global config, settings, extension restore
5 주차	Verify + Report	검증 명령, Markdown/HTML report, manual task queue
6 주차	QA 및 self-dogfooding	실제 맥 간 migration 리허설, known issues 목록

6.2 작업 우선순위

- 가장 먼저 manifest 와 restore phase 를 고정한다. UI 보다 데이터 계약이 우선이다.
- 그 다음 Homebrew export/import 와 dotfiles 복원을 만든다. 여기까지 되면 전체 가치의 절반 이상이 생긴다.
- 그 후 Git global, VS Code, verify report 를 붙인다.
- 마지막으로 resume, 충돌 처리, UX polishing 을 진행한다.

6.3 추후 확장 후보

후보	추가 이유	복잡도
JetBrains 어댑터	개발자 IDE 다양성 확대	중간
asdf / nvm / pyenv 지원	언어 런타임 재현성 향상	중간
패키지 암호화	로컬 package 유출 위험 감소	중간
Docker / DB exporter	개발 서비스 상태까지 확장	높음
APFS snapshot 보조 export	일관된 파일 시점 확보	높음 [R9]

7. 리스크와 결정 로그

7.1 주요 리스크

리스크	영향	대응
도구별 설치 방식 차이	복원 불일치	어댑터별 검증과 manual task 설계
교차 architecture 문제	prefix 및 package availability 차이	preflight 경고, 동일 architecture 우선 검증
권한 제한	일부 경로 export 실패	v1 권한 최소화, blocked state 명시 [R8]
패키지 버전 변화	완전 동일 재현 어려움	manifest 에 버전 기록, best-effort 문구 명시
secret 유출 가능성	보안 사고	기본 제외, 마스킹, opt-in 분리

7.2 지금 내려도 좋은 결정

- v1 은 서버 없이 로컬 전용으로 간다.
- v1 IDE 지원은 VS Code 1 종으로 제한한다.
- v1 의 secret 정책은 '기본 제외'다.
- v1 공식 목표는 동일 architecture 안정화이고, 교차 architecture 는 best-effort 다.
- 보고서 포맷은 Markdown 우선, 필요 시 HTML 추가 생성으로 간다.

7.3 아직 미결정으로 남겨도 되는 항목

- 패키지 압축 형식(zip vs folder) 최종 선택
- v2 에서 암호화 패키지를 넣을지 여부
- JetBrains support 의 구체적 UX
- APFS snapshot 기반 export 를 실제로 사용할지 여부

8. 부록

8.1 개발 시작 체크리스트

- manifest schema 와 sample manifest 를 코드 저장소에 커밋
- HomebrewAdapter 와 DotfilesAdapter 인터페이스 정의
- target machine preflight 결과 모델 작성
- overwrite 정책과 checkpoint 저장 형식 확정
- 실제 본인 맥 기준 allowlist 초안 작성

8.2 권장 초기 저장소 구조

```
apps/
  MacEnvMover/
    App/
    Features/Export/
    Features/Import/
    Features/Verify/
    Domain/
    Infrastructure/Adapters/
      Homebrew/
      Dotfiles/
      Git/
      VSCode/
    Infrastructure/Checkpoint/
    Reports/
docs/
  product-doc-pack.docx
  manifest.schema.json
samples/
  manifest.sample.json
```

8.3 참고자료

[R1]	**Apple Support. Transfer to a new Mac with Migration Assistant. Updated Feb 10, 2026. support.apple.com/en-us/102613**
[R2]	Homebrew Documentation. Homebrew Bundle, brew bundle and Brewfile. docs.brew.sh/Brew-Bundle-and-Brewfile
[R3]	Homebrew Documentation. Installation. docs.brew.sh/Installation
[R4]	Homebrew Documentation. Cask Cookbook. docs.brew.sh/Cask-Cookbook
[R5]	Git Documentation. git-config. git-scm.com/docs/git-config
[R6]	Visual Studio Code Documentation. Command Line Interface. code.visualstudio.com/docs/configure/command-line
[R7]	Apple Support. Set up iCloud Keychain to autofill information on Mac. support.apple.com/guide/mac-help/mh43699/mac
[R8]	Apple Platform Security. Controlling app access to files in macOS. support.apple.com/guide/security/secddd1d86a6/web
[R9]	Apple Support. View APFS snapshots in Disk Utility on Mac. support.apple.com/guide/disk-utility/view-apfs-snapshots-dskuf82354dc/mac